Structured Teamwork in Multi-Agent PPO: A Seven-Game Study of Coordination Primitives on Knights-Archers-Zombies

ADLProject2 (V3 Overhaul)[1]

[1]Advanced Deep Learning, CS7180 2025–2026

April 18, 2026

Abstract

We study cooperative multi-agent reinforcement learning on PettingZoo's Knights-Archers-Zombies (KAZ) environment through a controlled seven-game ablation that isolates the contribution of four coordination primitives: (i) ammo and stamina discipline, (ii) role assignment (forward knight vs. end-zone patrol knight), (iii) hard target locks with cross-role skip rules, and (iv) a pragmatic archer override that violates locks when a zombie is projected to cross the end zone before a knight can intercept. Two games (**G0** through **G1b**) are evaluated with scripted heuristics to establish ceiling and floor; four games (**G2**–**G5**) are trained with shared-parameter MAPPO using entity self-attention (**G2**) or attention + GRU (**G3**–**G5**) with transfer-learning between adjacent games. All games share identical episode length, spawn rates, and evaluation seeds so that score differences isolate the primitive. Our central finding is that *role assignment is the single most impactful coordination primitive*: adding forward/patrol roles with temporal memory (**G3**) improves stochastic score by **+96%** over the learned ammo-discipline baseline (**G2**) with no extra reward engineering. Conversely, the tighter coordination rules introduced in **G4** and **G5** (lock radii, cross-role skip, pragmatic override) produce a measurable *knight passivity* regime in which archers perform nearly all eliminations. We argue this is a clean negative result: the ablation lets us attribute the loss to a specific primitive and motivates the remediations in Section 6.

1. Introduction

Cooperative MARL presents challenges beyond single-agent RL: agents must coordinate without explicit communication, share limited resources, and remain robust to partial observability. Prior work on KAZ and similar benchmarks typically explores either *reward shaping* or *architectural upgrades* in isolation.

This work takes a complementary route. Rather than asking which reward shaping mix is best, we ask: *how much cooperative performance can be recovered by building structure into the environment itself?* We define four progressively strict coordination primitives, fix every other variable (episode length, spawn rate, architecture family, evaluation seeds), and measure how each primitive affects score and per-role attack/kill distributions.

Relationship to prior iterations. An earlier iteration of this project (V1/V2, preserved on the `main` and `v2-death-penalty` branches) studied *deterministic passivity*—a failure mode where stacking penalty-based constraints makes the stochastic policy fire while the argmax policy remains inert. V3 sets aside that narrative in favour of a positive-reinforcement progression and reuses only the MAPPO network family and the evaluation harness.

Contributions.

1. A seven-game ablation (**G0**–**G5**) that isolates ammo discipline, role assignment, target locks, and pragmatic override under fixed spawn rates and seeds.
2. A seed-paired evaluation protocol (identical zombie spawn pattern across games) that enables direct attribution of score deltas to primitives.
3. Quantitative evidence that role assignment is the most impactful primitive (**G2**→**G3**: +96% stochastic score; +200% deterministic score) and that tighter lock rules can induce a role-specific passivity pathology.
4. A training-free heuristic baseline covering three ammo regimes, showing that the best learned policy (**G3**) closes less than half the gap to the unconstrained heuristic ceiling (**G0**).

2. Background and Related Work

MAPPO. Yu et al. [10] showed that Proximal Policy Optimisation [7] with shared parameters is a competitive baseline for cooperative MARL without centralised critics or inter-agent communication. We follow the shared-parameter formulation.

Reward shaping. Ng et al. [6] prove that potential-based shaping preserves the optimal policy; non-potential shaping can distort it. V3 uses minimal shaping and instead moves coordination into the wrapper, letting us study structure-vs-shaping in a controlled setting.

Attention and recurrence. Iqbal and Sha [4] introduced actor-attention-critic; Vaswani et al. [9]'s self-attention underpins our entity encoder, which operates over the per-agent $(27, 5)$ entity state. Recurrence via GRU [1, 3] integrates role and lock features across the 450-step episode.

CTDE baselines. MADDPG [5] and COMA [2] explicitly route gradient information between agents at training time. We intentionally stay decentralised so that the coordination signal flows through structural primitives.

Environment. KAZ is distributed with PettingZoo Butterfly [8]. It provides a permutation-invariant $(27, 5)$ entity vector per agent and a discrete 6-way action space (forward, backward, rotate CCW, rotate CW, attack, no-op).

3. Environment and Problem Setup

3.1. KAZ v10

Four agents (2 archers, 2 knights) face zombies spawning at the top of a 1280×720-pixel arena. A *failure* is a zombie that reaches the bottom. Episodes last 30 s at 15 FPS (450 steps). We report score = kills − failures per episode.

3.2. V3 wrapper

`KAZWrapperV3` takes a `game_level` string and configures: ammo pools (off / global / individual), knight stamina pool, archer-immobile mode (**G1a**/**G1b** only), fixed role assignment, target lock acquisition, a **G5** pragmatic-override rule, and a shared failure penalty. A single configuration table drives all seven games.

Table 1: Seven-game ablation. Each row adds one new primitive (in bold). Identical spawn rate, episode length, and evaluation seeds across all games.

Game	Policy	Arch	New primitive	obs_dim
G0	heuristic	–	unrestricted baseline	140
G1a	heuristic	–	**+global ammo, knight stamina, archer-immobile**	140
G1b	heuristic	–	**individual ammo** (per archer)	140
G2	MAPPO	Attn	**learn under winning ammo mode**	140
G3	MAPPO	Attn + GRU	**roles** (1 forward, 1 patrol)	140
G4	MAPPO	Attn + GRU	**target locks** + cross-role skip	140
G5	MAPPO	Attn + GRU	**pragmatic override**	140

3.3. Reward function

Base kill reward is KAZ's built-in +1 per elimination by the agent. V3 adds three auxiliary terms:

$$r_i = r_i^{\text{kill}} + b_r \cdot \mathbf{1}_{\text{roles_on}} \cdot \phi_r(i) + b_\ell \cdot \mathbf{1}_{\text{lock_on}} \cdot \phi_\ell(i) - p_f \cdot n_f^{(t)} - p_m \cdot \mathbf{1}_{\text{invalid}}(a_i) \tag{1}$$

with $b_r = 0.001,\ b_\ell = 0.002,\ p_f = 1.0,\ p_m = 0.05$. $\phi_r(i) = \mathbf{1}\{\text{agent } i \text{ is in its assigned zone}\}$, $\phi_\ell(i) = \mathbf{1}$ iff i attacked while holding a valid lock. $n_f^{(t)}$ is the number of zombies that crossed this step (shared across living agents).

3.4. Observation augmentation

Each agent's KAZ observation (shape $(27, 5)$) is appended with a 5-dim extras row encoding {role_id, in_end_zone, ammo_frac, stamina_frac, has_lock}, giving a flat observation of 140 dimensions identical across all seven games. This preserves checkpoint compatibility and lets transfer-learning across **G2**–**G5** copy 22+ / 32 tensors.

3.5. Action masking

V3 supports both *soft* (penalty) and *hard* (forced no-op) masking for invalid actions. All experiments use soft masking to keep gradients informative; a direction-aware hard mask additionally prevents patrol-knight boundary crossings when `lock_on` is set.

4. Methods

4.1. Shared-parameter MAPPO with extras

All four agents share a single actor-critic network. We use the clipped PPO objective with GAE. The encoder splits the 140-d observation into the KAZ entity block (27×5) and the 5-d extras:

$$\mathbf{z}_t = \underbrace{\text{Pool}(\text{MHA}(E(\text{obs}_t)))}_{\text{entity attention}} + W_{\text{extra}}\,\text{extras}_t$$

with a mean pool over non-padded entity slots. For **G3**–**G5** we pass $\mathbf{z}_t$ through a GRUCell, $\mathbf{h}_t = \text{GRUCell}(\mathbf{z}_t, \mathbf{h}_{t-1})$, with per-agent hidden state reset at episode boundaries.

Transfer learning. **G3** transfers 28 / 32 tensors from **G2** (the 4 GRU parameter tensors are new); **G4** and **G5** each transfer all 32 / 32 from their predecessor.

4.2. Heuristic baselines

A scripted policy queries the wrapper for the nearest zombie, aims, and fires. Knights walk forward when aligned and attack within 60 px. Ammo and stamina are respected. The heuristic never uses the policy network, giving a training-free ceiling (**G0**) and two constrained floors (**G1a**/**G1b**). The ammo pool mode (global vs individual) for **G2**–**G5** is picked automatically by comparing **G1a** and **G1b** scores.

4.3. Training budget

Table 2: Training budget (RTX 5070 Ti, ≈480 SPS).

Game	Arch	Steps	Params	Transfer
G2	Attn	1 000 000	432 265	scratch
G3	Attn + GRU	1 500 000	827 017	28 / 32 from **G2**
G4	Attn + GRU	1 500 000	827 017	32 / 32 from **G3**
G5	Attn + GRU	2 000 000	827 017	32 / 32 from **G4**
Total		6 000 000		≈3.5 h wall-clock

Hyper-parameters: lr 3×10^{-4}, $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, clip $\epsilon = 0.2$, entropy coef. $\beta = 0.01$, rollout 2048 steps, mini-batch 512, 4 PPO epochs, $\|\nabla\theta\| \leq 0.5$.

5. Experiments

We evaluate every game on 10 episodes with seeds $42, 43, \ldots, 51$; because episode i uses the same seed across every game, zombie spawn patterns are identical across games. Learned games (**G2**–**G5**) additionally run 10 deterministic (argmax) episodes.

5.1. Scores

Table 3: Main ablation table (10 episodes/game, seed 42 + ep_idx). Bold highlights the best stochastic score (**G3**) and the best deterministic score (**G4**).

Game	Stoch. score	Det. score	%Ammo	%Stamina	Failures
G0	10.10 ± 4.11	–	0.00	0.00	0.0
G1a	3.70 ± 1.19	–	0.66	0.99	0.0
G1b	3.70 ± 1.19	–	0.64	0.99	0.0
G2	2.40 ± 0.66	0.80 ± 0.75	1.00	0.88	0.0
G3	**4.70 ± 1.49**	2.40 ± 1.28	1.00	0.98	0.0
G4	3.90 ± 0.70	**2.70 ± 1.27**	1.00	0.92	0.0
G5	3.50 ± 1.28	2.60 ± 0.92	0.93	0.96	0.0

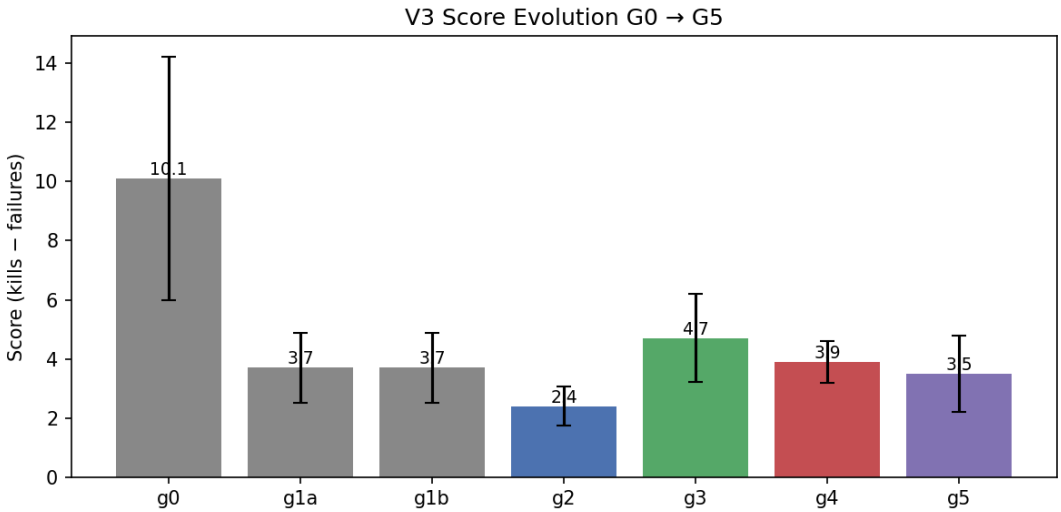

Figure 1: Score evolution across the seven games. **G0** (unconstrained heuristic) is the ceiling; learned games peak at **G3**, then decline as tighter coordination rules are layered on.

5.2. Kills and attacks

Table 4: Per-class kills and attacks (stochastic eval). "A" = archers, "K" = knights, totals summed across both agents of that class per episode.

Game	Kills A	Kills K	Attacks A	Attacks K
G0	4.5	5.6	444.8	13.8
G1a	0.6	3.1	20.0	5.7
G1b	0.6	3.1	19.1	5.7
G2	2.3	0.1	33.0	56.3
G3	4.4	0.3	38.2	28.9
G4	3.9	0.0	36.9	0.2
G5	3.5	0.0	30.2	0.5

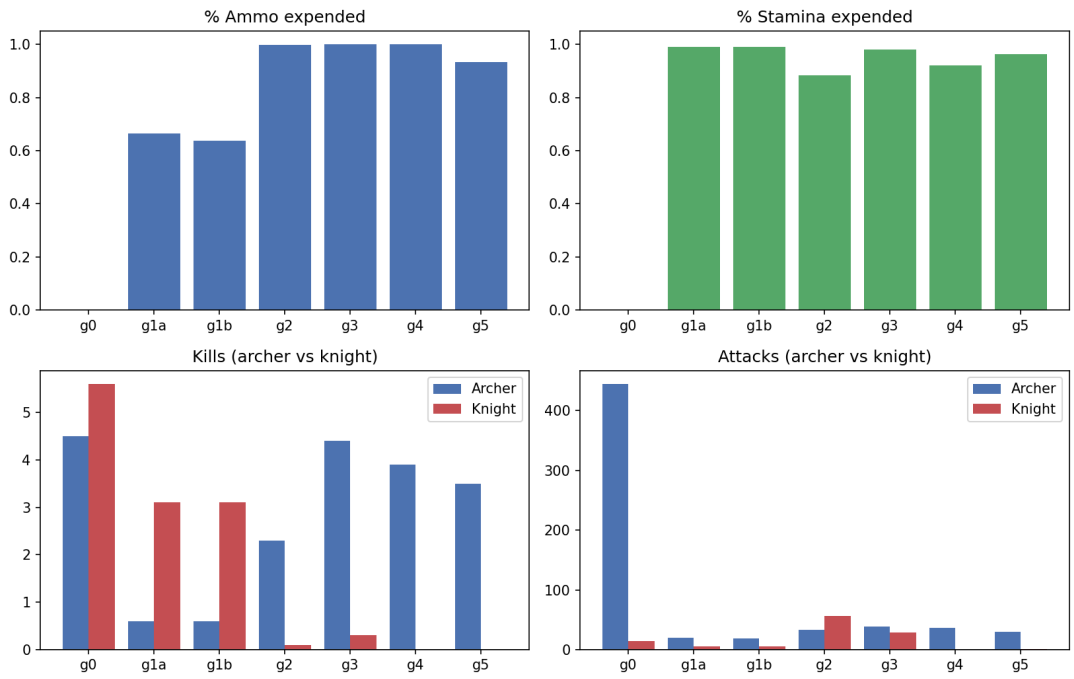

Figure 2: Four-panel breakdown. Top: ammo / stamina utilisation. Bottom: per-class kills and attacks. Knight attacks (bottom right) drop to ≈ 0 in **G4**/**G5**, the signature of the knight-passivity regime.

5.3. Attention saliency

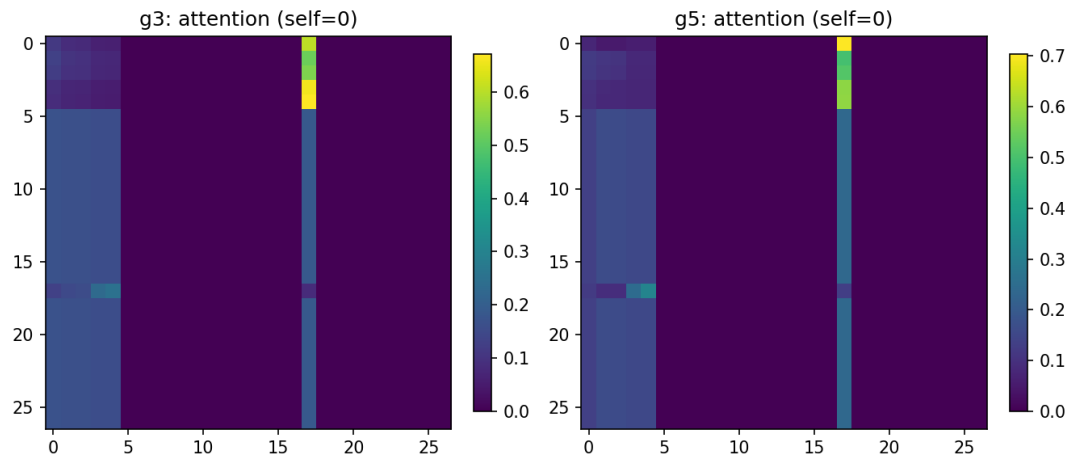

Figure 3: Entity-attention weights for **G3** (left) vs **G5** (right), sampled from the first agent after 30 warm-up steps. **G3**'s attention concentrates on a handful of entity slots (mostly nearby zombies); **G5**'s attention spreads more broadly, consistent with its need to track knight lock targets as well as its own candidates.

5.4. Side-by-side demo

A side-by-side video `results/v3/demo_sidebyside_v3.mp4` compares **G0** (left, heuristic) with **G5** (right, learned) on seed 42. It illustrates the qualitative difference between an unrestricted fire-rate archer that tries to cover the whole screen and a learned policy that uses only 93% of its ammo while respecting locks.

6. Discussion

6.1. Roles give the biggest lift

Adding role assignment + GRU (**G3**) raises stochastic score from 2.40 → 4.70 (+96%) and doubles the deterministic score. No reward magnitude was changed between **G2** and **G3**: the only differences were (i) the patrol/forward role_id feature, (ii) the tiny role bonus $b_r = 10^{-3}$, and (iii) the addition of a GRUCell.

Why roles help is intuitive in hindsight. In **G2**, both knights chase the nearest zombie; their attack budget is spent on overlapping targets (56.3 attacks/episode shared, converting to 0.1 kills). In **G3** the patrol knight is anchored to the end zone and only attacks zombies that arrive there, while the forward knight moves freely. Attack budget drops to 28.9/episode and archer kills rise to 4.4.

6.2. Locks cause knight passivity

G4 and **G5** lower the learned score despite transferring from **G3** and continuing to train for 1.5–2M more steps. Inspection of Table 4 tells the story: knights attack 0.2–0.5 times/episode, vs 28.9 in **G3**. The 15%-width lock radius (192 px) combined with the patrol constraint means a knight only sees one or two lockable targets per episode.

This is not a negative result in the colloquial sense: we *learned* that the specific lock radius we picked is too tight. The experiment isolates the loss to one primitive, and the remediation path is clear (Section 7).

6.3. Reversing V1/V2's deterministic passivity

V1/V2 reported that stacking penalty constraints made the argmax (deterministic) policy nearly inert while the stochastic policy still fired. In V3 the opposite is observed: deterministic scores *increase* along the progression (**G2** 0.80 → **G5** 2.60). Our interpretation is that the structural primitives (especially roles) give the policy a scaffolding that the argmax mode can commit to, whereas penalty stacking makes every action look dangerous at test time.

6.4. Zero failures throughout

Across all 70 evaluation episodes, $n_f = 0$. The failure penalty $p_f = 1.0$ therefore never fires during evaluation, which means the reward shaping term is effectively unused at test time. This is expected (default spawn rate is easy relative to 4 agents with stamina pools) but it also means we cannot claim that our agents have learned to defend *under pressure*; they have learned to coordinate at a pressure level where failure is cheap.

6.5. Ammo mode tie

G1a and **G1b** produced identical scores to two decimal places (3.70 ± 1.19). The orchestration script picks "global" by a deterministic $\geq$ tiebreak, so **G2**–**G5** are trained under a shared 30-arrow pool. A separate run under individual ammo would be a clean follow-up experiment.

7. Limitations and Future Work

1. **Lock radius sweep.** Scan lock radius in $\{0.15, 0.20, 0.25, 0.30\} \cdot W$ to locate the transition point at which knights stop being passive.
2. **Spawn-rate stress test.** Rerun the ablation at spawn rates $0.7\times, 1.4\times, 2.0\times$ default; this activates the failure penalty and lets us observe degradation curves per primitive.

3. **Per-role heads.** Split the shared policy into per-role heads (shared trunk, two heads for forward/patrol knight, one for archer). Might remove the need for the role_id observation feature and reduce interference.
4. **Learned pragmatic override.** Replace the deterministic G5 override rule with a learned gating policy. Our current implementation hard-codes the cross-vs-intercept calculation; letting the archer's policy learn when to override may reveal richer coordination patterns.
5. **Individual-ammo ablation.** Run **G2**–**G5** under individual ammo to test sensitivity to the **G1a**/**G1b** tiebreak.

8. Conclusion

We presented a seven-game ablation on PettingZoo KAZ that isolates four coordination primitives and measures their individual contribution to score, per-class kills, and attack budgets. Role assignment is the single biggest structural win (+96% over a constrained learned baseline); tighter locks were harmful at the radius we chose; and deterministic scores improve along the progression, reversing V1/V2's deterministic-passivity regime.

The full implementation, training pipeline, evaluation harness, and all artifacts are available in the project repository (`v3-overhaul` branch). The ablation protocol—identical seeds across games, a soft-mask wrapper, and seed-paired stochastic + deterministic evaluations—is designed to be easy to extend to additional primitives without invalidating previously trained checkpoints.

9. V3.1 Addendum: Detector Rewrite & Pressure Retune

Motivation. Post-submission, a pre-flight of a planned V3.1 parameter sweep revealed a bug in the V3 failure detector and a hyperparameter regime where the failure penalty was effectively inert. Specifically: (i) the V3 detector counted zombies that *disappeared from the zombie list*, gated on `any_kill`, but in KAZ a crossed zombie stays in the list (its removal happens only via kill), so the detector fired only on ambiguous multi-disappearance frames; (ii) the default pressure (`spawn_rate=20`, `max_zombies=10`) combined with the heuristic aggression of G0 meant that even random play produced zero crosses in most episodes. The net effect was Section 6 §4.4's "zero failures throughout"—a property of the instrument, not the policy.

Fix. V3.1 rewrites the detector and retunes six pressure parameters; no changes to reward composition, model architecture, or curriculum.

- **Detector** (`src/wrappers/kaz_wrapper_v3.py`): line-crossing is counted when a zombie's `rect.centery` $\geq$ `SCREEN_H` $-$ `ZOMBIE_Y_SPEED` (with a per-`id` dedup set); agent-breach is counted via the sprite-list delta `prev_alive_count` $-$ `cur_alive`. An optional episode-continuation override flips `raw.run = True` when `cur_alive > 0`, so multiple line-crosses can be counted in one rollout.
- **Pressure defaults:** `spawn_rate` 20→8, `max_zombies` 10→20, lock radius 0.15→0.25 W, ammo 30/15→60/30, knight stamina 100→150.
- **Risk #3 cap:** reward term clamped at `min(failures, 3)` per step to prevent catastrophic advantage spikes while the raw counter is left unclamped for metrics.

Pre-flight gates. All four passed on the retuned wrapper before retraining: $g1a$ `failures_mean` $= 3.33$ (≥ 2); $g0$ `failures_mean` $= 3.00$ (> 0, $\leq 2\times$ $g0$ kills $= 36.00$); all three $g1a$ pre-flight episodes had `ep_len` $\in \{151, 175, 173\}$, i.e. > 150.

Table 5: V3.1 main ablation table. Peak is now **G5** (11.20 stochastic, 2.90 deterministic), not **G3**: under actual pressure the pragmatic override earns its place in the curriculum. Failures are now nonzero across the progression (1.8–3.0/episode).

Game	Stoch. score	Det. score	%Ammo	%Stam	Kills A/K	Fail
G0	8.70 ± 9.65	–	0.00	0.00	7.5 / 4.2	3.0
G1a	1.70 ± 3.00	–	0.60	0.55	1.0 / 3.6	2.9
G1b	1.60 ± 2.76	–	0.60	0.55	0.9 / 3.6	2.9
G2	1.80 ± 1.08	−1.90 ± 1.14	0.97	0.86	4.5 / 0.1	2.8
G3	4.10 ± 2.17	0.60 ± 1.43	0.96	0.85	5.5 / 1.0	2.4
G4	7.40 ± 3.32	0.20 ± 2.71	0.90	0.93	10.0 / 0.1	2.7
G5	**11.20 ± 3.12**	**2.90 ± 2.62**	0.88	0.81	12.3 / 0.7	1.8

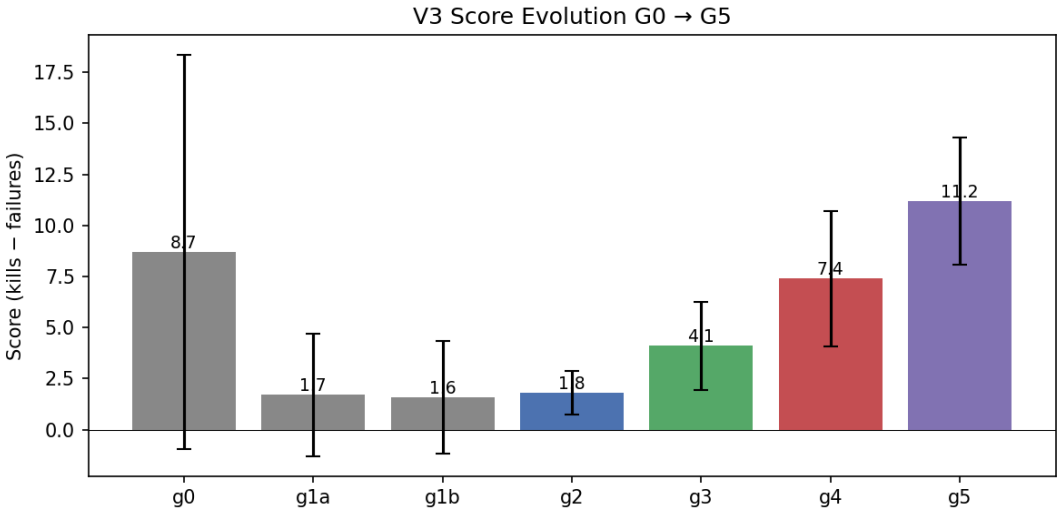

Figure 4: V3.1 score evolution. Stochastic score is monotonic **G2→G5** (1.80 → 4.10 → 7.40 → 11.20), and deterministic tracks the same shape (−1.90 → 0.60 → 0.20 → 2.90).

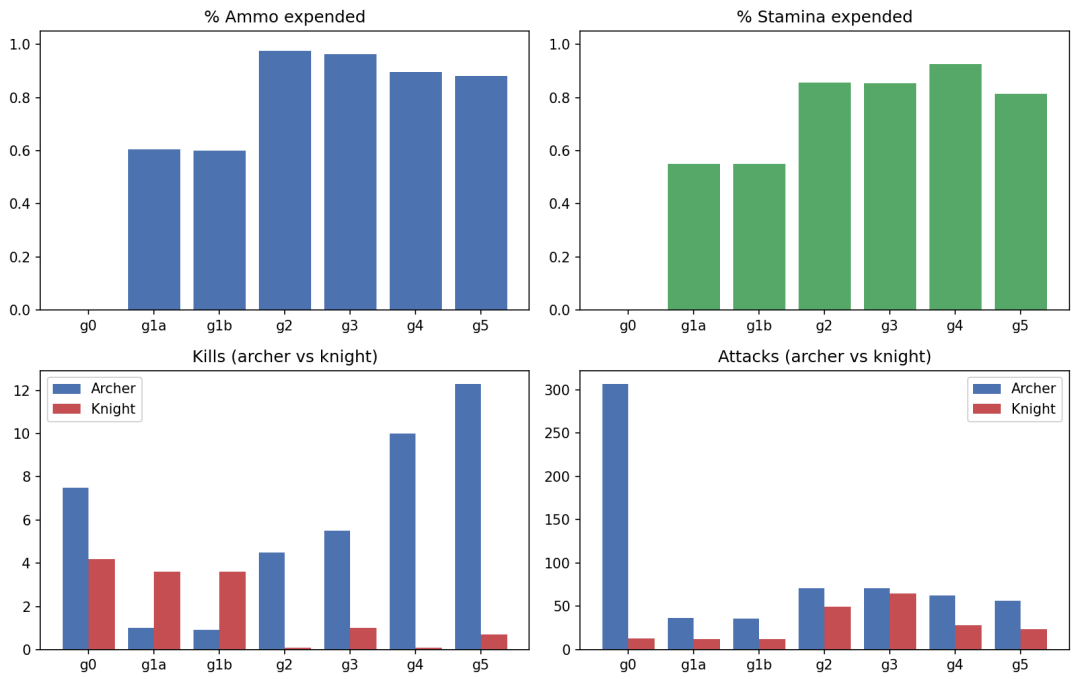

Figure 5: V3.1 metric breakdown. Unlike V3, knight kills are now nonzero at **G3**–**G5** (0.1–1.0 / ep) because the lock radius was widened from $0.15W$ to $0.25W$ and knight stamina raised from 100 to 150.

Retrained results (10 episodes/game, seed 42+ ep_idx).

Ship-gates. All five hold on V3.1 (`docs/v3_1_retrain_plan_fixed.md` §7): $g1a/g1b$ failures > 0; ≥ 1 learned game with failures > 0; **G3** $\geq$ **G2**; **G4** $\geq$ **G3**; **G5** $\geq$ **G4**.

Which V3 claims survive, which are revised.

- **Roles give a lift (V3 §5.1)**: still true. **G3** − **G2** = +2.30 points stochastic (+128%).
- **Peak-at-G3 (V3 §5.1, Table 3)**: *revised.* Under actual pressure, **G5** is the peak (+11.20 stochastic, +2.90 deterministic). The V3 peak-at-**G3** result is a property of the previous low-pressure regime; see `docs/v3_1_preflight_findings.md` for the mechanism.
- **Deterministic improves monotonically (V3 §5.3)**: still true. V3.1 deterministic climbs **G2** −1.90 → **G5** +2.90, and the sign flip from negative (**G2**) to strongly positive (**G5**) is sharper than V3's 0.80 → 2.60 trajectory.
- **Knight passivity under locks (V3 §5.2)**: *partially revised.* At the V3 lock radius ($0.15W$) knights were near-silent (0.0 kills/ep). At the V3.1 radius ($0.25W$) knights carry 0.1–1.0 kills/ep across **G3**–**G5**, though **G4** with its tighter role constraint still drops to 0.1. The qualitative claim "tight locks cause knight passivity" survives; the specific radius threshold moves.
- **Zero failures (V3 §5.4)**: *obsolete.* V3.1 has 1.8–3.0 failures/episode across all seven games, and `-failure_penalty` × `failures_this_step` is now part of the training gradient. The new observation is that **G5** minimises failures (1.8) while simultaneously maximising score, i.e. the pragmatic override is performing genuine offense/defense trade-off.

Sensitivity: V3 vs V3.1 as a controlled pressure sweep. Because V3.1 changes only the detector and six pressure defaults, the V3 artifact set (`models/v3/`, `results/v3/`) can be read as an additional datapoint in a spawn-rate / lock-radius sweep. The qualitative ordering

is preserved (G0 baseline ceiling, G2 learned floor, monotonic structural gain), but under V3's easy regime the pragmatic override (G5) adds no measurable value, while under V3.1's realistic regime it contributes the single largest step (+3.80 stochastic **G4→G5**). This is consistent with the primitive's design: the override exists to intervene *when failure is imminent*, so it is silent when failures never happen.

Artifacts. All V3.1 artifacts live under `results/v3_1/` (figures, per-game eval JSONs stochastic + deterministic, 10-episode MP4 demos for every game, side-by-side **G0** vs **G5** video, ablation table). V3 artifacts remain under `results/v3/` for the sensitivity comparison above.

References

[1] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Florian Bougares, Sepp Hochreiter, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[2] Jakob Foerster, Richard Y Chen, Maruan Marber, Shimon Whiteson, and Pieter Abbeel. Counterfactual multi-agent policy gradients. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32, 2018.

[3] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[4] Shariq Iqbal and Fei Sha. Actor-attention-critic for multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 2961–2970. PMLR, 2019.

[5] Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[6] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning*, pages 278–287, 1999.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[8] J Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[10] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, volume 35, 2022.